

A Novel Low Latency Datastore for the Edge

Muhammad Jazlan*

Lahore Univ. of Management Sciences
Lahore, Punjab
24100022@lums.edu.pk

Mughees ur Rehman†

Lahore Univ of Management Sciences
Lahore, Punjab
24100086@lums.edu.pk

ABSTRACT

Computing at the edge has been a key development in networks over the past decade. However, in the mobile computing space, it is still an emerging paradigm with a multitude of use cases in IoT, video analytics, gaming etc. The key idea in edge computing is to reduce computational requirements on device by offloading compute to the edge. In this project, we aim to solve the problems incurred during stateful application migration using a proposed 5G architecture by creating a novel low-latency data store to facilitate state migration on the edge. We develop a new data structure, a hashed queue, and use it to build a datastore. This datastore can be integrated in any 5G control plane design. We also test and evaluate our data structure, eventually implementing a datastore in Python and C++. Our evaluations show that the algorithm removes the need for sorting, and provides a 50% reduction in the payload transferred in the best case. We also discuss local and wide area replication schemes.

1 INTRODUCTION

5G and the next generation of cellular networks aim to support latency sensitive applications like virtual and augmented reality [5], connected autonomous vehicles [2, 4, 8], edge-based video analytics [3], and online gaming [13]. This class of latency sensitive applications has a very low delay budget to the order of 10's of milliseconds, e.g., AR/VR headsets have to maintain a latency of less than 17 seconds to make the app usable, and prevent side effects like nausea.

While the cloud can support such compute-intensive applications, it is not a golden bullet. The network has inherent limitations, e.g., the cellular Radio Access Network (RAN), the distance of the client from the cloud center. As a result, the applications need to be deployed physically closer to the user. According to the 5G white paper, we can place this application at any point between the user and the cloud center [1]. For the purpose of this report, we will assume for simplicity that our application is deployed at the first hop base station. The architecture is very flexible to changes in deployments and works just as well if the application deployment moves further away.

*Currently enrolled at Univ. of California, Davis

†Currently enrolled at Virginia Tech

This Multi Access Edge Computing (MAEC) paradigm requires a lot of support from the cellular core to function with negligible delays. If the delay budgets of applications are not met, they face significant downtime, resulting in a poor experience for the user.

A recently proposed 5G architecture, EdgeCat, reduces the app downtime by making the 5G control plane app-aware. This is achieved by introducing new components to the 5G architecture, including but not limited to a Base Station Predictor module, App-aware control plane processing and two-step app state synchronisation. The Base Station Predictor module provides mobility hints which allow us to determine whether a handover will take place, at least 100ms ahead of time. This is a very useful hint which can allow us to preemptively carry out actions to improve the performance of edge applications.

In this project, we asked the following questions:

- What are the main bottlenecks when using an architecture like *EdgeCat*?
- Is there anything we can do to reduce these bottlenecks?
- What quality of life improvements can we provide to developers (such as new APIs)?
- What are the consistency requirements?
- How much delay is not too much delay?
- How can we improve fault tolerance?

In this report, we discuss our motivation and provide some background on the 5G architecture. Then we answer each of these questions comprehensively, providing multiple options for solutions. We design and test our datastore with 2 different implementations, and finally we discuss future directions and improvements for our work.

2 BACKGROUND AND MOTIVATION

2.1 EdgeCat

EdgeCat is a recently proposed novel 5G architecture which aims to provide support for near-real time applications. EdgeCat consists of four major innovations:

- (1) EdgeCat introduces a new app-aware cellular control plane which exposes new APIs to edge apps, allowing the control plane to prioritize their control-plane processing.

- (2) EdgeCat stores a user's state in a datastore and exposes APIs to the application
- (3) A Target Base Station Prediction module is used, which receives real-time signals from the Radio Network Information (RNI) exposed to applications in 5G. This allows us to predict the destination base station ahead of an actual handover
- (4) The mobility handler module acts as a translation layer between the control plane and the application, providing information about the target edge host and application instance information to the source application.

These innovations allow EdgeCat to perform app-aware 5G handover and proactive state migration while providing read-your-writes consistency. EdgeCat reduces both the time required to process a latency sensitive application in the control plane, and the time required for application state migration.

Limitations: Upon analysis, we notice that there is room for improvement in the state migration process. The datastore used for the benchmarks is a simple wrapper over Redis, using counters as a way of tracking update frequency. This can be problematic for high loads since computation resources at the edge are vastly lower compared to a data center. As a result, we need to come up with a way of tracking frequency-based changes to reduce the impact of sorting.

Apart from this, during migration, a lot of this meta-state is transferred with the objects, resulting in a greater network usage. As the amount of dynamic state increases, frequent handovers (which occur approximately every 70 seconds in walking scenarios) can result in a large cost to the network operator.

We also note that this type of datastore is prone to data loss under failure as it does not do explicit state backup.

2.2 Low Latency Datastores

We conducted a survey of some of the state of the art open-source low latency data stores. In addition to understanding how they work, we tried to list down areas of improvements and some shortcomings for our specific use case. We have listed our findings below

2.2.1 Redis: Redis [11] is an open source, state of the art low latency key-value datastore.

Benefits:

- Redis offers low latency and high throughput, making it ideal for real-time applications.
- It supports state migration through its replication and persistence features.
- Data can be replicated to slave nodes or persisted to disk and restored on another instance.

Limitations

- Although Redis offers state migration but starts migrating once the device is completely disconnected which means higher payload at the time of migration.

2.2.2 Memcached: Memcached [9] is a high-performance, distributed memory object caching system.

Benefits:

- Can be easily distributed across multiple servers to scale horizontally.
- Optimized for quick read and write operations, providing low latency.

Limitations:

- Memcached does not support state migration as it lacks persistence and replication features necessary for migrating state between instances.

2.2.3 Zookeeper: Zookeeper [7] is a centralized service for maintaining configuration information, naming, providing distributed synchronization, and providing group services.

Benefits:

- Ensures strong consistency through consensus algorithms like ZAB (Zookeeper Atomic Broadcast).
- Designed to provide distributed synchronization and coordination services.

Limitations:

- Can introduce latency in large-scale deployments due to consensus overhead.
- Can be resource-intensive in terms of network and disk I/O due to frequent state synchronization.

2.2.4 PortKey : PortKey [10] is a distributed KV store that dynamically adapts data placement according to time-varying client mobility and data access patterns. **Benefits:**

- Optimal strategy predetermines the ideal placement for each KV over a short (near-instantaneous) time window.
- Incorporating knowledge of client mobility and data access patterns into KV placement decisions can substantially improve overall datastore performance (i.e., client-perceived request latencies)

Limitations:

- Lack of a customized MIGRATE API

2.3 Stateful Edge Applications

5G networks are designed to support a large number of edge applications through Mobile Edge Computing (MEC). These applications can be either stateless or stateful. Stateful applications depend on the state information previously reported by the client or generated by the server. For example, an edge

server that keeps a count of packets received from a client is a stateful application. When a user moves, the application state, such as the packet count, must be transferred to the new edge server to maintain accuracy. Real-world stateful edge applications are often more complex, requiring low latency and the ability to handle user mobility.

A few examples of such apps are

- CarMap [2]
- ORBSLAM [8]
- Mobile VR [5]
- Online Gaming [13]

2.4 Application Handovers

Discuss frequency, and limitations of EdgeCat for only being able to detect a handover at least 100ms before the actual time. We change the limitation where we assume that we have 100ms. We assume that handover can take place shortly after we get a hint, well before the 100ms mark.

3 DESIGN

After concluding our study of current solutions, we came up with a list of challenges and design goals

3.1 Challenges

Managing data migration within the datastore involves efficiently tracking the usage frequency of each key. The system used in EdgeCat maintains a counter that increments with every key usage, such as during 'put' operations, and records the timestamp of the last update. These metrics are used to calculate the update frequency rate of each key over a given time period.

Upon receiving a Mobility Hint, the datastore sorts the keys based on their update frequency rates, which incurs a computational complexity of $O(n \log n)$. Post sorting, the datastore prioritizes key migration starting with the least frequently used keys and ending with the most frequently used keys.

While this approach of **Primitive Feedback Solution** ensures accurate ordering for a small number of keys, it presents significant drawbacks as the number of keys increases:

- **Computational Cost:** Sorting keys becomes a substantial bottleneck with a complexity of $O(n \log n)$.
- **Performance Impact:** For applications managing a large volume of keys, the sorting process can significantly slow down the migration API.
- **Scalability Issues:** The efficiency of sorting deteriorates with larger datasets, leading to potential performance issues.

This method assumes a scenario where the number of keys remains relatively small to maintain efficiency. We aim

to implement a datastore that provides tunable consistency and which works for different variety of Edge Applications in future.

3.2 Design Goals

To address the inefficiencies of the **primitive feedback solution**, we propose the **Rapid Feedback Queue**, which aims to optimize resource utilization and improve the efficiency of data migration between base stations. The primitive solution incurs significant network overhead when each PUT and DELETE request is forwarded to a new base station after a mobility hint is received. In high-request scenarios, this overhead can lead to network congestion. The main problem in this is that on each migration we need to use the sorting based approach which requires a complexity of $O(n \log n)$ where n are number of keys.

In our design we aim to implement a single queue system using a doubly linked list to maintain the order of keys. Whenever a PUT or GET operation occurs for a key, it is added to the head of the queue. Over time, the least frequently updated keys will naturally migrate towards the tail of the queue. Periodically, keys from the tail are extracted and migrated, allowing us to manage data migration efficiently. Additionally, a limit can be set to migrate the first n key-value pairs in the queue during subsequent requests.

This approach ensures reduced network overhead and eliminates the need for computationally expensive sorting algorithms. The system can scale both geographically and locally. Geographically, it can handle data migration and resource allocation across multiple regions, and locally, it can manage a large volume of keys and user operations within a single cell tower efficiently.

In terms of consistency we aim to provide tunable consistency levels similar to the distributed datastore *Manu* [6], ensuring that data remains consistent across different base stations while allowing flexibility based on application needs.

4 THE RAPID QUEUE ALGORITHM

4.1 The Algorithm

In the Rapid Feedback Queue approach, which utilizes a doubly linked list, we categorise keys into two distinct states for migration purposes: "Cold" for less frequently used keys and "Hot" for more frequently used keys. This categorisation helps us prioritize the migration of "Cold" keys before "Hot" ones.

To differentiate between the two, we define the "Hottest" key as the one most recently used, while the "Coldest" key is the one least recently used. The queue is organized such that its head points to the "Coldest" key and its tail to the "Hottest" key.

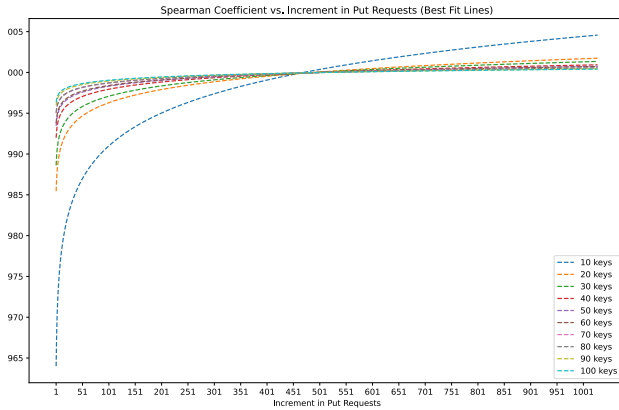


Figure 1: The Spearman Rank Correlation Coefficient calculated for keys sorted in ascending order of insertion

A simple Queue data structure is used for insertion and deletion of the keys.

- The algorithm maintains the order of insertion on a given node. It will always keep track of keys that have **most recently** been used.
- This algorithm does not need any sorting, saving us massive amounts of algorithmic runtime, by removing the overhead of sorting on every single insertion, or a batch insertion.
- We can do an in-order traversal of the keys in the list, allowing us to generate new kinds of meta information (we haven't explored this yet).

Some drawbacks of using this approach are listed below:

- We increase the amount of memory used. In addition to the value (from the KV pair), we are also keeping track of nodes, which is in the form of pointers. We calculate that an additional 24 bytes would be used per node (3 additional 8-byte values per node: two pointers, and an 8-byte key). However, the algorithmic memory complexity still remains in $O(n)$.

4.2 Big Oh Analysis - Vanilla Queue

If we conduct a Big O analysis of this solution, we end up with the same problem as comparison based solutions. Here are the run times:

- Insertion - $O(n)$ - Each insertion (GET/PUT) has to perform a lookup in the queue to see where the current key is, remove it from that location, and insert it at the head

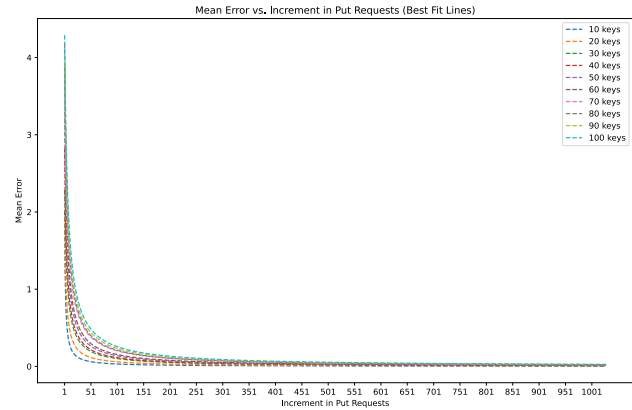


Figure 2: The Mean Absolute Error calculated for keys sorted in ascending order of insertion

- Deletion - $O(n)$ - Each deletion has to perform a lookup in the queue to see where the current key is, and remove it.
- Dequeue - $O(1)$ - Since it is a doubly linked list, we can remove each node in $O(1)$

4.3 A Rapid Queue Solution

In order to reduce the lookup overhead, while still maintaining the order given to us by a queue, we propose to index list nodes based on the keys they contain. This is only introducing one level of indirection, in which we are ensuring that we maintain one key property, i.e., all operations should take constant time $O(1)$. In addition, there are a number of other benefits to hashed approach, which are listed below:

4.4 Big Oh Analysis - Rapid Queue

If we conduct a Big O analysis of this solution, we end up with:

- Insertion - $O(1)$ - Each insertion (GET/PUT) has to perform a lookup in the queue. We use a hash map for search which utilizes $O(1)$ to land on the current key is, remove it from that location, and insert it at the head.
- Deletion - $O(1)$ - Each deletion has to perform a lookup in the queue to see where the current key is, and remove it.
- Dequeue - $O(1)$ - Since it is a doubly linked list, we can remove each node in $O(1)$

4.5 Correctness

Our approach to the Rapid Feedback Queue considers the MRU scheme instead of MFU. In evaluating the effectiveness of our "Most Recently Used" (MRU) method against the "Most Frequently Used" (MFU) strategy, we aimed to ascertain the degree of correlation between MRU ordering and true frequency-based ordering.

4.5.1 Methodology. To achieve this, we conducted experiments using a set of defined parameters. The parameters included:

- The number of keys involved.
- The update frequency for each key, for instance, with 10 keys, the first key is updated once, the second key is updated twice, the third key thrice, and so on.

We aimed to measure the deviation of our MRU approach from the MFU's true order. For this comparison, we employed the Spearman rank coefficient, which quantifies the degree of association between two ranked variables. Moreover, we also tried to find the mean error between the MRU queue and MFU in order to see how far we are from true ordering.

4.5.2 Experiment. We created a set of key-value pairs and applied the put operation to different keys, varying the frequency of updates in line with their designated update rates. These operations were executed in a completely random sequence to evaluate the system's performance under such conditions.

4.5.3 Results. Our findings indicated a strong alignment with the MFU scheme. Specifically, we observed that as the update frequency between successive keys widened — particularly when the frequency difference was about an order of magnitude, around 10 to 12 times — our MRU approach demonstrated over 95% similarity to the MFU ordering. This suggests that our datastore is likely to perform optimally when there is a significant variance in the update frequencies across different keys. Our results are shown in Figures 1 and 2.

The findings indicate that a greater disparity in the frequency of 'put' requests allows us to approximate an ordering that mirrors the 'Most Frequently Used' sequence starting from the 'Most Recently Used' data. A Spearman rank coefficient nearing 1 suggests a high degree of similarity between the two lists, meaning the ranking of each item in our list closely matches its counterpart in the 'Most Frequently Used' list.

The analysis also reveals that as the update frequency disparity between keys widens, the mean absolute difference in ordering between our 'Most Recently Used' (MRU) method and the 'Most Frequently Used' (MFU) benchmark

diminishes, indicating a convergence toward the true MFU sequence.

4.5.4 Assumptions. The datastore is designed with the expectation that the application running on the base station will exhibit a pattern involving a considerable number of keys, which are subject to widely varying update frequencies. However, the system may not be able to ensure a reliable ordering if the update frequencies of the keys are too closely aligned.

5 BENCHMARKS

5.1 Microbenchmarks

We conducted microbenchmark evaluations to compare the performance of our datastore's PUT and GET API with Redis [11], known for its high-speed in-memory data storage capabilities. Redis was chosen as a benchmark due to its reputation for rapid data operations. Our experiments showed that our datastore performed equally well in terms of speed and efficiency.

To comprehensively analyze the performance, we plotted the number of keys on the x-axis and the time taken for operations on the y-axis. Initially, we encountered issues with socket delays impacting the timing results. To address this, we explicitly added timestamps within the Redis code, modifying it to capture the actual time taken for GET and PUT operations.

Furthermore, we ensured that both databases were completely flushed before each test to maintain a consistent starting point. To achieve more accurate and fair results, we performed multiple iterations of each test and calculated the average time taken for the operations.

Additionally, we discovered that a Redis SET request involves two commands: a GET and a PUT operation, whereas a GET request only involves a GET operation. This distinction helped us better understand the underlying mechanics and provided more accurate benchmarking data.

The benchmark results have been shown in figures 4 and 3

5.2 Migration Time

One of the primary goals of this project was to reduce the overall time for migration. Figures 5 shows the comparison of the migration time in comparison to Redis. On the face of it, the results seem like they are not promising. However, we need to keep in mind that Redis is written using an asynchronous event loop in C. The Rapid Queue prototype was built using a synchronous model in Python. By our estimates, there is an approximately 30-40% room for improvement with a simple change.

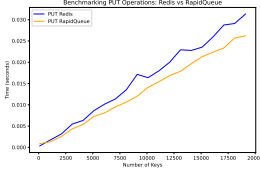


Figure 3: Benchmarking PUT Operations: Redis vs RapidQueue

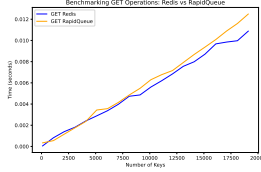


Figure 4: Benchmarking GET Operations: Redis vs RapidQueue

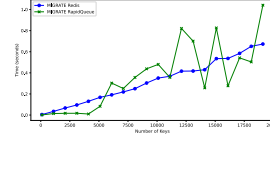


Figure 5: Migration Times Comparison

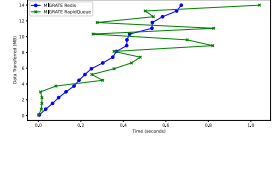


Figure 6: Data Transferred Comparison

Figure 6 shows a comparison of the total time taken to transfer data using our algorithm compared to Redis. The graph shows the total size of the KV pairs vs the time it takes to migrate these KV pairs. We can see that even while using a synchronous model, the data points for Rapid Queue are above those for Redis. The randomness in the graph can be explained by the synchronous nature of the Rapid Queue prototype.

6 FAULT TOLERANCE AND CONSISTENCY

One important discussion surrounding datastores is consistency and fault tolerance. Applications can have different requirements. Some may require strong consistency while others may require weaker consistency [12]. In our datastore, consistency is not straightforward. It depends on the state of the Rapid Queue, which is built using insertions over time. Hence, consistency here means maintaining the order of insertions. As a natural consequence, the weakest consistency we can provide is sequential. In this section, we discuss in detail how our datastore can manage consistency.

6.1 The Serializer

Our assumption about the deployment leaves very few choices for us to serialize our updates. One choice is to use a consensus algorithm like Raft for replicas within an edge node. However, it would be extremely time consuming and overkill in this case.

Upon closer inspection, we realized the following:

- If the edge instance of the application crashes, all dynamic state on the application instance is lost
- Due to the highly dynamic nature of the application, we assume that even for a few minutes of downtime, the state would need to be reconstructed, or would be stale anyways
- Any communication with the user happens through the edge application instance

- The user never directly accesses the datastore, rather it is the edge application that manages the state

Drawing from this analysis, we note that the edge application instance contains all state and is critical for operation. As a result, it can act as a serializer. For this particular scenario, the edge application being a single point of failure is not a problem, as discussed above.

We use a simple serialization scheme with n replicas. We also provide a tuneable parameter b with the constraint

$$b > \lceil \frac{n}{2} \rceil$$

. This constraint is crucial for maintaining the sequential consistency requirement that arises due to the inherent nature of the Rapid Queue Algorithm. If $b = n$, then we have strong consistency.

An example scenario is described below:

- (1) The current sequence number is 10, $n = 5$, $b = 3$ (sequential consistency)
- (2) The edge app instance receives an update. It sends the following message to all n replicas '`<seq>,PUT,A,1`' and waits for b replicas to reply.
- (3) After receiving a response from the first b replicas, the edge app receives another update.
- (4) The sequence number increments to 11. The message is now '`<seq>,PUT,A,2`'.
- (5) The edge app receives a response from 2 replicas that did not previously acknowledge the update (let's say they were down) and 1 response from a replica that acknowledged the previous update. In this instance, we have at least 1 replica with the latest state.
- (6) Next, the edge app receives a GET request. It sends the following message '`GET,<key>`'. Now we wait for any b replicas to respond. Given our majority requirement, at least one replica will have seen the latest update and return '`A,2`'

As discussed in our algorithm above, the nature of Rapid Queue results in a need to process updates sequentially. If a replica is missing an earlier sequence number, it cannot

process subsequent updates. Therefore, we need to have a handoff mechanism, where the serializer can provide the info of the replica with the missing sequence number. This scheme does not introduce much overhead at the edge application's end, since it can simply keep track of the cumulative updates.

Replicas will only communicate with each other if there are any missing values to avoid any in-cast congestion due to rapidly failing nodes. In this case, the response to a GET request will be modified to include the latest sequence number a replica has seen, the last sequence number it has processed, and the value associated with that sequence number. From b replies, we will see the highest sequence number received. Using this sequence number, we will decide whether we need replicas to communicate withing themselves.

To summarise, we provide a tuneable parameter to the developer who can choose to have strong consistency or sequential consistency with robust fault tolerance.

6.2 Local and Wide Area Replication

The replication scheme described in 6.1 can be further modified to improve response times. In the scenario above, if 3 out of 5 replicas are placed off-site, then the response time will increase by orders of magnitude due to transmission delays. As a result, we can modify our scheme to use two different parameters, n and w , where n is the number of local replicas and w is the number of off-site replicas. $total = w + n$, however, since we want to improve our response times, we will have two different majority quorums, nb and wb .

7 FUTURE WORK

Non serialization based ordering for a Rapid Queue algorithm

We believe that the introduction of the Rapid Queue algorithm can be used for a variety of new use cases, including building quick lookup caches. Some other notable directions of work include

- Non Serialization based ordering schemes for fault tolerance and replication
- Another promising direction is using multiple Rapid Queues to make distinct categorizations between how dynamic a certain key is and using one queue for each category
- As observed in section 5.2, we observe that performance matches state of the art, while reducing the overall time required to transfer state. Therefore we believe that Rapid Queues can also be used in general purpose datastores to reduce bandwidth consumed by backup and synchronization algorithms.

8 CONCLUSION

In this paper we tried to create a datastore that would help edge applications manage their state better without the developer having to manually manage state. We introduced, prototyped and benchmarked a new algorithm, Rapid Queue, and compared it to the current state of the art low latency datastore Redis. We used Spearman Rank Correlation and Mean Absolute Error to validate the hypothesis 'RapidQueue can provide a good estimation of frequency-based sorting'. We used our Python prototype to compare the operations to Redis, achieving very similar performance in microbenchmarks. We also compared the total time taken to transfer data and observed a reduction in total time at higher payload sizes.

REFERENCES

- [1] 5G-PPP. 2023. Innovation Trends in I4.0 Enabled by 5G and beyond Networks. (Oct 2023).
- [2] Fawad Ahmad, Hang Qiu, Ray Eells, Fan Bai, and Ramesh Govindan. 2020. CarMap: Fast 3D Feature Map Updates for Automobiles. (2020), 1063–1081 pages. <https://www.usenix.org/conference/nsdi20/presentation/ahmad>
- [3] Ganesh Ananthanarayanan, Paramvir Bahl, Peter Bodík, Krishna Chintalapudi, Matthai Philipose, Lenin Ravindranath, and Sudipta Sinha. 2017. Real-Time Video Analytics: The Killer App for Edge Computing. *Computer* 50, 10 (2017), 58–67. <https://doi.org/10.1109/MC.2017.3641638>
- [4] Ali J. Ben Ali, Zakieh Sadat Hashemifar, and Karthik Dantu. 2020. Edge-SLAM: Edge-Assisted Visual Simultaneous Localization and Mapping. In *Proceedings of the 18th International Conference on Mobile Systems, Applications, and Services (MobiSys '20)*. Association for Computing Machinery, New York, NY, USA, 325–337. <https://doi.org/10.1145/3386901.3389033>
- [5] Mohammed S. Elbamby, Cristina Perfecto, Mehdi Bennis, and Klaus Doppler. 2018. Toward Low-Latency and Ultra-Reliable Virtual Reality. *IEEE Network* 32, 2 (2018), 78–84. <https://doi.org/10.1109/MNET.2018.1700268>
- [6] Rentong Guo, Xiaofan Luan, Long Xiang, Xiao Yan, Xiaomeng Yi, Jigao Luo, Qianya Cheng, Weizhi Xu, Jiarui Luo, Frank Liu, Zhen-shan Cao, Yanliang Qiao, Ting Wang, Bo Tang, and Charles Xie. 2022. Manu: A Cloud Native Vector Database Management System. (2022). [arXiv:cs.DB/2206.13843](https://arxiv.org/abs/2206.13843)
- [7] Patrick Hunt, Mahadev Konar, Yahoo ! Grid, Flavio P Junqueira, Benjamin Reed, and Yahoo ! Research. [n. d.]. *ZooKeeper: Wait-free coordination for Internet-scale systems*. https://www.usenix.org/legacy/event/atc10/tech/full_papers/Hunt.pdf
- [8] Raúl Mur-Artal and Juan D. Tardós. 2017. ORB-SLAM2: an Open-Source SLAM System for Monocular, Stereo and RGB-D Cameras. *IEEE Transactions on Robotics* 33, 5 (2017), 1255–1262. <https://doi.org/10.1109/TRO.2017.2705103>
- [9] Rajesh Nishtala, Hans Fugal, Steven Grimm, Marc Kwiatkowski, Herman Lee, Harry C. Li, Ryan McElroy, Mike Paleczny, Daniel Peek, Paul Saab, David Stafford, Tony Tung, and Venkateshwaran Venkataramani. 2013. Scaling Memcache at Facebook. (2013), 385–398 pages. <https://www.usenix.org/conference/nsdi13/technical-sessions/presentation/nishtala>
- [10] Joseph Noor, Mani Srivastava, and Ravi Netravali. 2021. Portkey: Adaptive Key-Value Placement over Dynamic Edge Networks. In *Proceedings of the ACM Symposium on Cloud Computing (SoCC '21)*. Association for Computing Machinery, New York, NY, USA, 197–213. <https://doi.org/10.1145/3472883.3487004>
- [11] Redis. 2023. Redis. (2023). <https://redis.io/>
- [12] Andrew S Tanenbaum and Maarten Van Steen. 2016. *Distributed systems : principles and paradigms*. [Netherlands] Maarten Van Steen.
- [13] Wuyang Zhang, Jiachen Chen, Yanyong Zhang, and Dipankar Raychaudhuri. 2017. Towards efficient edge cloud augmentation for virtual reality MMOGs. In *Proceedings of the Second ACM/IEEE Symposium on Edge Computing (SEC '17)*. Association for Computing Machinery, New York, NY, USA, Article 8, 14 pages. <https://doi.org/10.1145/3132211.3134463>

A THE RAPIDQUEUE ALGORITHM

All problems in computer science can be solved by another level of indirection

In order to reduce the lookup overhead, while still maintaining the order given to us by a queue, we propose to index list nodes based on the keys they contain. This is only introducing one level of indirection, in which we are ensuring that we maintain one key property, i.e., all operations should take constant time ($O(1)$).

In addition, we use a doubly linked list, which means that we can do point insertions and deletions in $O(1)$

A.1 Insert Algorithm

We create the following the algorithm for insertion

```
Store ← PUT KEY VALUE
Hash ← h(KEY)
Store[KEY] = Node(KEY, VALUE, next
Add to Linked List O(1)
```

A.2 Delete Algorithm

We create the following the algorithm for insertion

```
Store ← PUT KEY VALUE
Hash ← h(KEY)
del Store[KEY]
Remove from Linked List O(1)
```

A.3 Enqueue

Since all insertions take $O(1)$, the enqueue function also takes $O(1)$

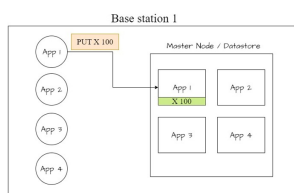
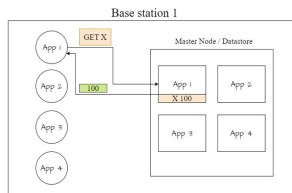
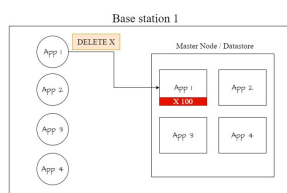
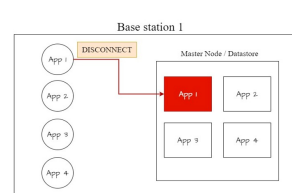
A.4 Dequeue

Since all deletions take $O(1)$, the dequeue function also takes $O(1)$

In addition, there are a number of other benefits to hashed approach, which are listed below:

- (1) The algorithm maintains the order of insertion on a given node. It will always keep track of keys that have **most recently** been used.
- (2) This algorithm does not need any sorting, saving us massive amounts of algorithmic runtime, by removing the overhead of sorting on every single insertion, or a batch insertion.
- (3) We can do an in-order traversal of the keys in the list, allowing us to generate new kinds of meta information.

Some drawbacks of using this approach are listed below:

**Figure 7: PUT operation****Figure 8: GET operation****Figure 9: DELETE operation****Figure 10: DISCONNECT operation**

(1) We increase the amount of memory used. In addition to the value (from the KV pair), we are also keeping track of nodes, which is in the form of pointers. We calculate that an additional 24 bytes would be used

per node (3 additional 8-byte values per node: two pointers, and an 8-byte key). However, the algorithmic memory complexity still remains in $O(n)$.